REGISTRO DE TECNOLOGÍA Y PROPUESTA INICIAL

Integrantes Grupo 3:

Adrián Quesada Jiménez

Camila Alpízar Alfaro

Emilio Calderón Jiménez

Gustavo Jiménez Hidaldo

Justin Zhu Fan

Salma Capín Romero

Caso asignado: Caso 2 - MercadoTico 360 (Marketplace de comercios costarricenses)

Modelo NoSQL asignado: Documentos

Tecnología: MongoDB

Versión prevista: MongoDB Atlas y MongoDB Compass

Para el entorno de trabajo, se utilizará un clúster gratuito en la nube (MongoDB Atlas) junto con MongoDB Compass (o extensión en VS Code) para la gestión del repositorio y el desarrollo colaborativo, evitando dependencias de instalación local complejas entre los integrantes del equipo.

Justificación de Correspondencia con el Modelo NoSQL Asignado

El caso de negocio MercadoTico 360 requiere solucionar las limitaciones de un modelo relacional tradicional al gestionar un catálogo con productos sumamente variados (alimentos, ropa, tecnología, artesanías y servicios).

El modelo orientado a documentos de MongoDB se justifica técnicamente por los siguientes factores:

1. **Esquema flexible (Catálogo heterogéneo):** Cada categoría de producto posee atributos distintos. En lugar de utilizar tablas relacionales con múltiples columnas nulas o tablas hijas para atributos dinámicos, el modelo de documentos (BSON/JSON) permite guardar productos con atributos personalizados según su categoría, eliminando campos vacíos y simplificando el almacenamiento.
2. **Documentos embebidos (Historial de pedidos):** Permite anidar las líneas de detalle directamente dentro del documento principal de cada pedido. Esto garantiza la integridad histórica de la compra sin depender de operaciones JOIN complejas entre múltiples tablas.

3. **Evolución del esquema sin interrupciones:** Permite incorporar nuevas categorías de productos o agregar nuevos datos a la estructura del pedido en tiempo real, facilitando la escalabilidad del marketplace a medida que crece.
4. **Capacidad de análisis:** MongoDB cuenta con un motor de agregaciones (Aggregation Framework) muy potente, idóneo para extraer reportes analíticos y métricas operativas requeridas por la plataforma.

Descripción de la arquitectura y estrategia de escala

La solución adopta una arquitectura orientada a la nube (Cloud Native) y un flujo de trabajo colaborativo distribuido entre los integrantes del grupo. La infraestructura se compone de tres capas principales:

1. **Capa de control de versiones y colaboración:**
   - Repositorio público de GitHub centralizado para el código del generador de datos, scripts de carga, consultas e índices.
   - Flujo de trabajo mediante ramas locales e integración por Pull Requests.

2. **Capa de almacenamiento y motor de datos:**
   - Clúster en MongoDB Atlas diseñado para soportar la escala mínima requerida: 25,000 productos, 10,000 clientes y 50,000 pedidos.
   - Trabajo remoto colaborativo sobre la misma instancia compartida en la nube.

3. **Capa de desarrollo y cliente:**
   - Uso de Visual Studio Code y MongoDB Compass conectados al clúster mediante una cadena de conexión (URI) segura.
   - Desarrollo de scripts de automización en Python para la generación masiva de datos sintéticos.

Estructuras principales de datos

Se proponen cuatro colecciones principales, balanceando el uso de documentos embebidos y referencias para cumplir los requisitos del caso:

Colección 1: productos – Catálogo heterogéneo (25 000 documentos)

- Estrategia: Documentos con esquema flexible que permiten almacenar diferentes atributos según la categoría del producto (alimentos, ropa, tecnología, artesanías, servicios, etc.)
- Estructura clave:
  - **_id**: Identificador único
  - **nombre, precio, categoria, comercio_id, stock**
  - **atributos** (objeto embebido variable): Permite representar atributos específicos por categoría sin forzar un esquema fijo ni dejar columnas nulas:
    - *Ropa: { talla: "M", color: "Azul", material: "Algodón" }*
    - *Tecnología: { garantia_meses: 12, voltaje: "110V" }*
    - *Alimentos: { fecha_vencimiento: "2026-12-01", requiere_refrigeracion: true}*

Colección 2: comercios (500 documentos)

- Estrategia: Documento individual referenciado.
- Estructura clave:
    - **_id:** Identificador del comercio
    - **nombre, correo, teléfono, categoría, estado**
    - **dirección** (objeto embebido): *{ provincia, canton, distrito, direccion_exacta }*

Colección 3: clientes (10 000 documentos)

- Estrategia: Documento individual referenciado.
- Estructura clave:
    - **_id**: Identificador del cliente
    - **nombre, correo, telefono, cedula**
    - **dirección** (objeto embebido): *{ provincia, canton, distrito, direccion_exacta }*

Colección 4: pedidos – Historial transaccional (50 000 documentos)

- Estrategia: Documentos embebidos.
- Estructura clave:
    - **_id**: Identificador del pedido
    - **fecha, cliente_id** (referencia al cliente), **estado** *("Pendiente", "Enviado", "Entregado"),* **monto_total**
    - **lineas_detalle** (arreglo embebido): Contiene el detalle de cada ítem comprado (*producto_id, nombre_producto, precio_unitario, cantidad, subtotal*)
    - Justificación de diseño: Al anidar las líneas de detalle dentro del documento del pedido, se garantiza la inmutabilidad del historial. Si un producto cambia de precio o se modifica posteriormente en el catálogo, la foto histórica del pedido se conserva intacta sin necesidad de JOINs complejos.

Propuesta inicial de índices y patrones de consulta

Para garantizar un rendimiento óptimo sobre la escala de datos, se prevé la creación de los siguientes índices iniciales:

- Índice compuesto en productos (categoria + precio): Para resolver de forma eficiente búsquedas de productos filtrados por categoría y ordenados por rango de precio.
- Índice en atributo específico (atributos.talla o atributos.garantia_meses): Permite realizar búsquedas rápidas sobre campos dinámicos embebidos dentro de las categorías.
- Índice en pedidos (cliente_id + fecha): Permite recuperar todos los pedidos de un cliente específico de forma inmediata.

Casos de uso principales e identificadores clave

Los principales casos de uso serán operaciones necesarias para consultar y administrar la información de MercadoTico 360. Por ejemplo, la solución propuesta se utilizará en:

- La búsqueda de productos por categoría, precio o atributos específicos.
- La consulta de productos asociados a un comercio específico.

- La recuperación de pedidos realizados por un cliente específico.
- La actualización del estado de pedidos específicos.

Los identificadores clave utilizados para relacionar los documentos serán los siguientes:

- **_id de producto:** Identifica a cada producto del catálogo.
- **_id de cliente:** Identifica al cliente relacionado con un pedido
- **_id de pedido:** Identifica a cada pedido realizado
- **_id de comercio:** Identifica al comercio que ofrece ciertos productos en el catálogo.

Plan de Pruebas

El plan de pruebas busca verificar que el modelo documental en MongoDB cumple los requisitos del caso MercadoTico 360 y medir cómo los índices propuestos mejoran el desempeño de las consultas frecuentes.

Datos y entorno de prueba

- Datos: sintéticos, generados con Python (Faker) y una semilla fija para que sean reproducibles: 25000 productos en 8 o más categorías, 500 comercios, 10000 clientes y 50000 pedidos con varias líneas de detalle.
- Entorno: MongoDB Server v7.0 sobre MongoDB Atlas, con Python y pymongo. Se documentarán el hardware y las versiones utilizadas al ejecutar las pruebas.

Pruebas de desempeño

- Búsqueda de productos: por categoría y rango de precio, y por categoría, precio y atributo específico (por ejemplo, talla), cada una sin índice y con índice. Se mide el tiempo, los documentos examinados y la etapa de ejecución (COLLSCAN frente a IXSCAN) con explain("executionStats").
- Pedidos de un cliente: recuperación de los pedidos de un cliente ordenados por fecha, sin y con el índice (cliente_id + fecha), con las mismas métricas.
- Agregaciones: ventas totales por categoría y ticket promedio, con el Aggregation Framework. Se mide el tiempo de ejecución.
- Carga y concurrencia: carga masiva de los datos y lecturas concurrentes con 10, 50 y 100 hilos. Se mide documentos por segundo, latencia media y percentil 95.

Pruebas funcionales

- Productos de distintas categorías conviven en la misma colección con sus atributos propios, sin campos nulos.
- Consulta de los productos asociados a un comercio específico.
- Actualización del estado de un pedido, que modifica solo ese campo.
- Si cambia el precio de un producto después de una compra, el pedido histórico conserva el precio original en sus líneas de detalle.

- Agregar un atributo o una categoría nueva no requiere migración ni afecta las consultas existentes.

Metodología

Cada prueba se ejecuta con una corrida de calentamiento y 30 repeticiones, y se reporta la mediana y el percentil 95. La medición sin índice se realiza antes de crear el índice, sobre los mismos datos. Los resultados se guardan en archivos CSV, junto con capturas de explain(), y la prueba se considera superada cuando las pruebas funcionales se cumplen y, con índice, el plan de ejecución usa IXSCAN y examina menos documentos que sin índice.

Riesgos técnicos identificados

- Credenciales de Atlas expuestas en el repositorio público.
    - Mitigación: usar un archivo .env incluido en .gitignore y un .env.example sin secretos, y cambiar toda contraseña que haya sido subida al repositorio.
- Entorno no reproducible si solo funciona con Atlas.
    - Mitigación: documentar claramente en el README la configuración del entorno, las variables necesarias y los pasos para reproducir la conexión y ejecución del proyecto.
- Límites del clúster gratuito de Atlas (almacenamiento y rendimiento) que pueden afectar las mediciones.
    - Mitigación: documentar el entorno de prueba y, si es necesario, repetir las mediciones en local.
- Líneas de pedido que no respeten la estructura definida (nombre del producto, precio unitario, cantidad y subtotal), lo que rompería la integridad histórica.
    - Mitigación: validar que el generador y las operaciones CRUD guarden esos campos y verificarlo con la prueba de cambio de precio.
- Índices que no cubran las consultas reales.
    - Mitigación: revisar con explain() y evaluar un índice compuesto que incluya el atributo consultado.
- Atributos inconsistentes entre productos de una misma categoría.
    - Mitigación: aplicar validaciones en Python para controlar la estructura esperada de los documentos y mantener consistencia en los atributos utilizados por categoría.
- No alcanzar el volumen mínimo de datos.
    - Mitigación: verificar los conteos de cada colección después de la carga.
- Mediciones poco confiables por caché, otros procesos o hardware distinto.
    - Mitigación: calentamiento, 30 repeticiones, mediana y percentil 95, y entorno documentado.
- Falla de la demostración en vivo.
    - Mitigación: respaldo local de los datos, capturas de pantalla y un video corto de la ejecución.